

5.7 Datoteke u C++

- 5.7.1 Modovi otvaranja datoteka u klasi ios
- 5.7.2 Pokazivač datoteke
- 5.7.3 Funkcijski članovi klase fstream za rad s datotekama
- 5.7.4 Tekstualne datoteke u C++
- 5.7.5 Datoteke s fiksnom veličinom sloga

5.7 Datoteke u C++

Datoteka predstavlja dinamičku strukturu podataka jer joj veličina (a u C++ ni struktura) nije unaprijed zadana. C++ tretira datoteke kao objekte klase *fstream*, odnosno klasa *ofstream* (za izlazne datoteke) ili *ifstream* (za ulazne datoteke) :

Primjer :

```
fstream dat("dat1.dat",ios::in | ios::out | ios::nocreate);
```

-otvara U/I datoteku dat("dat1.dat"), tako da ne kreira novu datoteku (tj. otvara postojeću)

5.7.1 Modovi otvaranja datoteka u klasi ios

- app (append) - upis na kraju datoteke
- ate (at-end) - otvaranje i skok na kraj datoteke
- binary - upis/učitavanje u binarnom modu
- in - otvara datoteku za čitanje
- nocreate - otvara postojeću datoteku
- noreplace - otvara nepostojeću datoteku
- out - otvara datoteku za upis podataka
- trunc - briše sadržaj postojeće datoteke

5.7.2 Pokazivač datoteke

Pokazivač datoteke pokazuje poziciju unutar datoteke gdje se upis/čitanje obavlja. Pozicija je izražena u cijelom broju bajtova od početka datoteke do mjesta upisa/čitanja. Klase *ostream* (uključujući *ofstream*) i *istream* (*ifstream*) koriste različite funkcijske članove za rad s pokazivačem datoteke (*fstream* može koristiti jedne i druge) :

ostream:

- tellp() - vraća trenutnu vrijednost pokazivača
- seekp() - pridružuje vrijednost pokazivaču (određuje poziciju slijedećeg upisa)

istream :

- tellg() - vraća trenutnu vrijednost pokazivača
- seekg() - pridružuje vrijednost pokazivaču (određuje poziciju slijedećeg čitanja)

5.7.3 Funkcijski članovi klase *fstream* za rad s datotekama

5.7.3.1 Funkcijski članovi *tellp* i *tellg* za očitavanje vrijednosti datotečnog pokazivača

```
<cjelobrojna_varijabla> = <datotečni_objekt>.tellg();
```

Primjer:

```
pozicija = dat.tellg();
```

5.7.3.2 Funkcijski članovi *seekp* i *seekg* za pridruživanje vrijednosti datotečnom pokazivaču

```
<datotečni_objekt>.seekg(pozicija_unutar_datoteke);
```

Primjer:

```
dat.seekg(5*sizeof(slog));
```

5.7.3.3 Funkcijski član *open* za otvaranje datoteke

```
<naziv_datotečnog_objekta>.open (<fizičko_ime_datoteke>,<mod_otvaranja>)
```

Primjer :

```
ofstream dat;  
dat.open ("Podaci.dat", ios::out | ios::binary); /* Otvara izlaznu datoteku  
“Podaci.dat” u binarnom načinu rada. */
```

5.7.3.4 Funkcijski član *read* za čitanje datoteke

```
<naziv_datotečnog_objekta>.read (<polje_znakova>,<veličina_polja_znakova>)
```

Primjer :

```
dat.read ( (char*) &slog, sizeof (slog) ); /*Čita strukturu slog iz datoteke dat*/
```

5.7.3.5 Funkcijski član *write* za upis sloga u datoteku

```
<naziv_datotečnog_objekta>.write (<polje_znakova>,<veličina_polja_znakova>)
```

Primjer :

```
dat.write ((char*)&slog,sizeof(slog));/*Upisuje strukturu slog u datoteku dat*/
```

5.7.3.6 Funkcijski član *eof()* – indikator kraja datoteke

```
<naziv_datotečnog_objekta>.eof ()
```

vraća logičku vrijednost :

- 0 – nije kraj datoteke
- 1 – datotečni pokazivač označava poziciju kraja datoteke

Funkcijski član *eof()* koristi se kod slijednog čitanja datoteke :

Primjer:

```
while (!(dat.eof())) {  
dat.read ((char *) &slog,sizeof(slog));
```

```
.  
}
```

5.7.3.7 Funkcijski član `close` za zatvaranje datoteke

```
<naziv_datotečnog_objekta>.close()
```

Primjer :

```
dat.close(); // Zatvara datoteku dat
```

5.7.4 Tekstualne datoteke u C++

Tekstualne datoteke sastoje se od redova teksta, s tim da svaki od redova završava kontrolnim znakovima CR (Carriage Return; ASCII kod 13; prebacuje kursor na početak reda) i LF (Line Feed; ASCII kod 10; pomiče kursor u slijedeći red). Zajedno, CR i LF čine znak EOLN - znak za kraj reda. Prema tome, pojedini zapisi (redovi teksta) mogu biti varijabilne dužine, što može biti prednost, obzirom na zauzeće prostora na disku, ali i nedostatak, jer je tada nemoguća prekoredna obrada (direktno pozicioniranje na pojedini zapis u datoteci), nego samo slijedna - da bi pročitati/upisali neki podatak, moramo najprije pročitati/upisati sve prethodne.

Primjer :

```
#include <iostream.h>  
#include <fstream.h>  
void main(){  
    fstream dat;  
    dat.open("tekst.txt",ios::out);  
    dat << "Ovo se upisuje u tekstualnu datoteku." << endl;  
    dat << "I ovo (u drugi red)" << endl;  
    dat << 150 << endl;  
    dat.close();  
}
```

Upis u tekstualnu datoteku odgovara konzolnom ispisu podataka na ekranu računala, što znači da se svi podaci konvertiraju u znakovne nizove.

Ako želimo čitati tekstualnu datoteku, možemo postupiti kao u slijedećem primjeru :

```
#include <iostream.h>  
#include <fstream.h>  
void main(){  
    char red[80];  
    fstream dat;  
    dat.open("tekst.txt",ios::in);  
    while(1){  
        dat.getline (red,sizeof(red),"n"); // čitanje reda iz datoteke  
        if (dat.eof()) break;  
        cout << red << endl;  
    };  
    dat.close();  
}
```

5.7.5 Datoteke s fiksnom veličinom sloga

Za razliku od tekstualnih datoteka, koje imaju varijabilnu dužinu zapisa, a svaki od zapisa sadrži odgovarajući graničnik (znak za kraj reda), datoteke s fiksnom veličinom sloga ne sadrže graničnike, a pojedini zapisi su istog, zadanog tipa, pa time i veličine. C++ omogućuje slijednu i prekorednu obradu datoteka s fiksnom veličinom sloga.

Primjer (upis podataka):

```
#include <iostream.h>
#include <fstream.h>
struct tslog{
    char prez[15],ime[15];
    int g_rod,g_stu;
};
void main(){
    tslog slog;
    char jos='d';
    fstream dat ("podaci.dat",ios::out,ios::binary);
    while ((jos=='d')||(jos=='D')){
        cout << "Prezime : "; cin >> slog.prez;
        cout << "Ime : "; cin >> slog.ime;
        cout << "Godina rođenja : "; cin >> slog.g_rod;
        cout << "Godina studija : "; cin >> slog.g_stu;
        dat.write ((char *) &slog,sizeof(slog));
        cout <<"Hocete li jos upisivati (d/n) ? ";
        cin >> jos;
    };
    dat.close();
}
```

Definiran je tip `tslog` i u datoteku su upisivani podaci isključivo tog tipa. Za upis podataka u datoteku korišten je funkcijski član `write`. Konverzija `(char *)&slog` potrebna je jer `write` radi s poljima znakova, odnosno, potrebno je memorijskom prostoru koji se zapisuje na disk dodijeliti odgovarajući (znakovni) tip.

Datoteku kreiranu u prethodnom primjeru možemo čitati kao u slijedećem primjeru :

```
#include <iostream.h>
#include <fstream.h>
struct tslog{
    char prez[15],ime[15];
    int g_rod,g_stu;
};
void main(){
    tslog slog;
    fstream dat ("podaci.dat",ios::in,ios::binary);
    while (1){
        dat.read ((char *) &slog,sizeof(slog));
        if (dat.eof()) break;
        cout << "Prezime : " << slog.prez << endl;
        cout << "Ime : " << slog.ime << endl;
        cout << "Godina rođenja : " << slog.g_rod << endl;
        cout << "Godina studija : " << slog.g_stu << endl;
    };
    dat.close();
}
```

}